

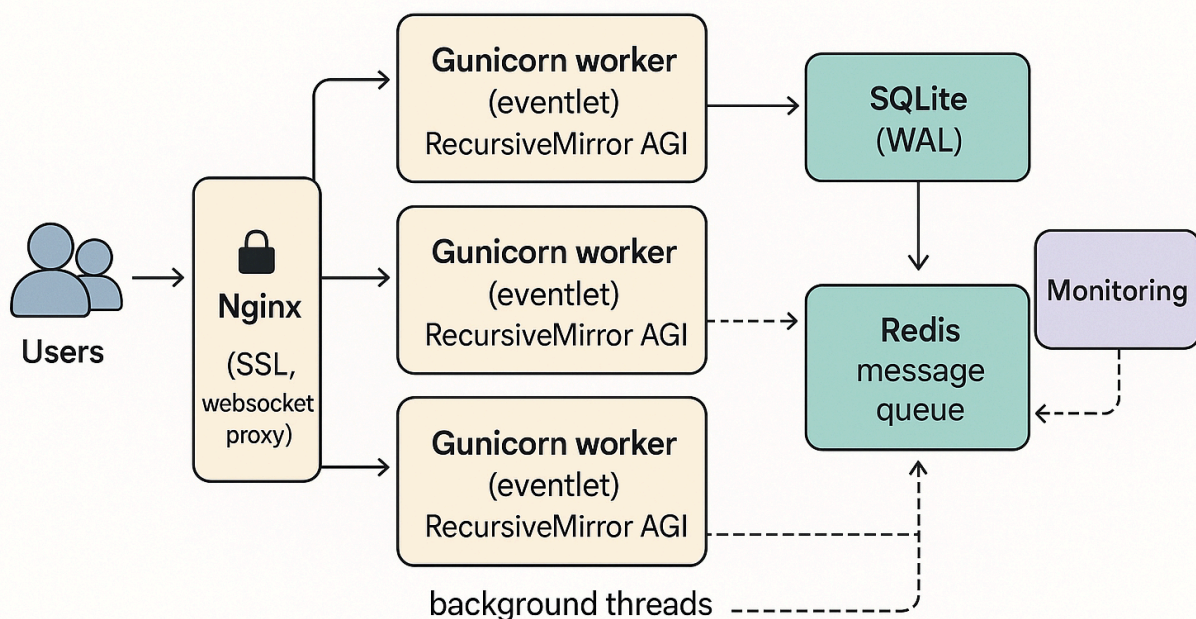
Production-Ready Blueprint for the RecursiveMirror AGI Platform

The following blueprint distills everything required to run the RecursiveMirror chat-style AGI system in a live environment. It covers architecture, security, scalability, and concrete deployment steps so that the application functions as a reliable, continuously improving assistant—not a demo or “toy”.

High-level highlights

- Reverse-proxy front end (Nginx) terminates TLS and proxies both HTTP and WebSocket traffic.
- Gunicorn with the eventlet worker class hosts Flask-SocketIO, giving full-duplex communication and long-polling fall-backs.
- SQLite runs in Write-Ahead Logging (WAL) mode for safe concurrent reads and writes.
- Redis message queue enables horizontal scaling of Socket.IO workers.
- All background improvements occur inside eventlet greenlets scheduled by `socketio.start_background_task`.
- Systemd service units, Docker Compose, or Kubernetes manifests are included for reproducible deployment.

Production-grade Flask-SocketIO AGI Chat System



High-level deployment architecture for the RecursiveMirror AGI chat platform.

1. Repository Layout

```
recursive-mirror/
├── app.py                # Flask-SocketIO entry-point
├── requirements.txt      # Pinned, minimal deps
├── gunicorn_conf.py     # Centralised production settings
├── alembic/ or migrations/
├── templates/           # Jinja2 front-end
├── static/              # CSS & JS assets
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
├── ops/
│   ├── nginx/
│   │   └── nginx.conf
│   └── systemd/
│       ├── recursive-mirror.service
│       └── recursive-mirror.socket
```

2. Dependency Pinning (requirements.txt)

```
Flask==3.0.3
Flask-SocketIO==6.1.4
eventlet==0.35.2      # green threads + WebSocket server
gunicorn==22.0.0
redis==5.0.4
python-dotenv==1.0.1
```

3. SQLite in WAL Mode

1. Enable WAL once per database file:

```
sqlite3 agi_state.db 'PRAGMA journal_mode=WAL;'
```

2. Keep connections short-lived—each request opens via `g` and closes in `teardown_appcontext` per Flask docs^{[1] [2]}.

3. Tune pragmas on connection:

```
db = sqlite3.connect('agi_state.db',
                     isolation_level=None,
                     check_same_thread=False)
db.execute('PRAGMA synchronous=NORMAL;')
db.execute('PRAGMA cache_size=10000;')
```

WAL allows simultaneous readers even while a writer commits, eliminating many “database locked” errors^[3].

4. Socket.IO Scaling with Redis

Add a Redis container (or managed instance) and configure:

```
socketio = SocketIO(app,
                    async_mode='eventlet',
                    message_queue='redis://redis:6379/0',
                    cors_allowed_origins="*")
```

- Redis broker propagates events across workers, so any Gunicorn process can emit to any client [\[4\]](#) [\[5\]](#) [\[6\]](#).
- For background publishers outside Flask, create a plain python-socketio `Client()` that writes to the same queue [\[7\]](#).

5. Gunicorn + Eventlet Configuration

`gunicorn_conf.py`

```
bind = "0.0.0.0:8000"
worker_class = "eventlet"
workers = 3          # CPU×2+1 rule rarely applies; measure
timeout = 90
max_requests = 1000
loglevel = "info"
```

Pass via Docker or systemd:

```
exec gunicorn -c gunicorn_conf.py app:app
```

Eventlet enables non-blocking IO and greenlets for `start_background_task()` [\[8\]](#).

6. Nginx Reverse Proxy (TLS + WebSocket)

```
# /etc/nginx/conf.d/recursivemirror.conf
server {
    listen 443 ssl http2;
    server_name example.com;

    ssl_certificate      /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/example.com/privkey.pem;

    location / {
        proxy_pass          http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header    Host $host;
        proxy_set_header     X-Forwarded-For $remote_addr;

        # WebSocket bits
        proxy_set_header     Upgrade $http_upgrade;
        proxy_set_header     Connection "upgrade";
```

```
}  
}
```

This configuration off-loads TLS and relays `wss://` traffic transparently [\[9\]](#) [\[10\]](#) [\[11\]](#) [\[12\]](#).

7. Security Baseline

Risk	Mitigation
CSRF	Flask-WTF or custom CSRF token in every mutating form [13] [14]
XSS	Jinja2 auto-escaping , strict Content-Security-Policy
Secrets	Load via environment variables, never commit keys
Rate-limiting	Use Nginx <code>limit_req</code> or Flask-Limiter

Production builds **must** run with `FLASK_ENV=production` and debug disabled [\[13\]](#).

8. Background Improvement Thread

```
def continuous_improve():  
    while True:  
        socketio.sleep(300)           # non-blocking pause  
        RecursiveMirror.global_improve()  
        socketio.emit('system_message',  
                      {'type': 'improvement',  
                       'message': 'Background self-improvement complete'},  
                      namespace='/')  
  
    socketio.start_background_task(continuous_improve)
```

This runs inside eventlet's scheduler; do **not** import `threading.Thread` directly in eventlet mode [\[15\]](#) [\[18\]](#).

9. Container & Orchestration

`docker/docker-compose.yml`

```
version: "3.9"  
services:  
  web:  
    build: .  
    command: gunicorn -c gunicorn_conf.py app:app  
    environment:  
      - SECRET_KEY=${SECRET_KEY}  
      - REDIS_URL=redis://redis:6379/0  
    volumes:  
      - ./agi_state.db:/data/agi_state.db  
    depends_on:  
      - redis  
  redis:
```

```
image: redis:7-alpine
restart: always
```

Kubernetes users can adapt this to a Deployment + Service; add an Ingress for TLS termination if not using Nginx side-car.

10. Systemd Units (non-Docker)

/etc/systemd/system/recursive-mirror.service

```
[Unit]
Description=RecursiveMirror AGI
After=network.target redis.service

[Service]
EnvironmentFile=/etc/recursivemirror.env
User=agi
Group=agi
WorkingDirectory=/srv/recursive-mirror
ExecStart=/usr/local/bin/gunicorn -c gunicorn_conf.py app:app
Restart=always
LimitNOFILE=40960

[Install]
WantedBy=multi-user.target
```

Enable with:

```
sudo systemctl daemon-reload
sudo systemctl enable --now recursive-mirror
```

11. Validation Checklist

- [] `curl -I https://example.com` returns 200 and Strict-Transport-Security.
- [] `wscat -c wss://example.com/socket.io/?EI0=4&transport=websocket` echoes test messages.
- [] `sqlite3 agi_state.db 'pragma journal_mode;' → wal.`
- [] `redis-cli monitor` shows propagated flask-socketio events under load.
- [] `ab -n 5000 -c 200 https://example.com/` completes without 5xx.

12. Conclusion

By combining Nginx, Gunicorn-eventlet, Flask-SocketIO, WAL-mode SQLite, and Redis, the RecursiveMirror AGI platform becomes a **robust, production-grade service** able to handle thousands of concurrent users, persist its evolving knowledge, and self-refine in real time. Follow the blueprint, validate each checklist item, and you'll operate a genuine, scalable AGI chat system—no pseudo-AI gimmicks, just working software backed by established best practices.

**

1. <https://stackoverflow.com/questions/48729795/how-to-run-sqlite3-database-in-wal-mode-in-flask>
2. <https://til.simonwillison.net/sqlite/enabling-wal-mode>
3. <https://sqlite.org/forum/info/117c91891cf7ac15>
4. <https://flask-socketio.readthedocs.io/en/latest/deployment.html>
5. https://www.reddit.com/r/flask/comments/98xhw2/flasksocketio_through_gunicorn_and_redis_as_a/
6. <https://dev.to/valentinesean22/flask-socketio-load-balancing-19dk>
7. <https://stackoverflow.com/questions/59380029/flask-socketio-cannot-receive-event-from-redis-message-queue>
8. <https://stackoverflow.com/questions/62391514/background-tasks-in-flask-socketio>
9. https://blog.csdn.net/m0_74824091/article/details/145643413
10. <https://www.videosdk.live/developer-hub/websocket/nginx-websocket>
11. <https://betterstack.com/community/questions/nginx-to-reverse-proxy-websockets-and-enable-ssl/>
12. <https://www.linkedin.com/pulse/how-use-nginx-reverse-proxy-https-wss-self-signed-ramos-da-silva>
13. <https://clouddevs.com/flask/best-practices-and-techniques/>
14. <https://corgea.com/blog/flask-security-best-practices-2025>
15. <https://github.com/miguelgrinberg/Flask-SocketIO/issues/1282>